Question 1: What is Ensemble Learning in machine learning? Explain the key idea behind it.

Answer:
Ensemble Learning is a technique in machine learning where multiple models (called *base learners* or *weak learners*) are combined to improve overall performance.

Key idea:
Instead of relying on a single model, ensemble methods combine predictions from several models to:

- Reduce errors
- Improve accuracy
- Increase robustness

Common approaches:

- Averaging (e.g., Bagging)
- Weighted voting (e.g., Boosting)

Question 2: What is the difference between Bagging and Boosting?

Answer:

Aspect	Bagging	Boosting
Goal	Reduce variance	Reduce bias & variance
Training	Independent models	Sequential models
Sampling	Bootstrap sampling	Reweighted data
Handling errors	No focus on mistakes	Focus on previous errors
Example	Random Forest	AdaBoost, Gradient Boosting

Question 3: What is bootstrap sampling and what role does it play in Bagging methods like Random Forest?

Answer:
Bootstrap sampling is a technique where samples are drawn **with replacement** from the original dataset.

Role in Bagging:

- Creates multiple different training datasets

- Each model is trained on a different bootstrap sample
- Introduces diversity among models
- Used in Random Forest to build multiple trees

Question 4: What are Out-of-Bag (OOB) samples and how is OOB score used to evaluate ensemble models?

Answer:
OOB samples are the data points **not included** in a bootstrap sample.

OOB Score:

- Acts like a validation set
- Used to evaluate model performance without separate test data
- Computed by predicting each sample using models that did not train on it

---

Question 5: Compare feature importance analysis in a single Decision Tree vs. a Random Forest.

Answer:

Aspect	Decision Tree	Random Forest
Stability	Low (high variance)	High (averaged)
Bias	Can overfit	More generalized
Importance	Based on single tree splits	Averaged across many trees
Reliability	Less reliable	More robust

Question 6: Write a Python program to:

- **Load the Breast Cancer dataset using sklearn.datasets.load_breast_cancer()**
- **Train a Random Forest Classifier**
- **Print the top 5 most important features based on feature importance scores.**

Answer:

```
from sklearn.datasets import load_breast_cancer

from sklearn.ensemble import RandomForestClassifier

import pandas as pd
```

```
# Load dataset
data = load_breast_cancer()
X = data.data
y = data.target
feature_names = data.feature_names

# Train model
model = RandomForestClassifier(random_state=42)
model.fit(X, y)

# Feature importance
importances = model.feature_importances_
feature_importance_df = pd.DataFrame({
    'feature': feature_names,
    'importance': importances
}).sort_values(by='importance', ascending=False)

# Top 5 features
print(feature_importance_df.head(5))
```

Sample Output:

```
                feature  importance
27  worst concave points    0.150
23       worst perimeter   0.120
20          worst radius   0.110
7     mean concave points  0.095
22         worst area     0.085
```

Question 7: Write a Python program to:

- **Train a Bagging Classifier using Decision Trees on the Iris dataset**

● **Evaluate its accuracy and compare with a single Decision Tree**

Answer:

```
from sklearn.datasets import load_iris

from sklearn.tree import DecisionTreeClassifier

from sklearn.ensemble import BaggingClassifier

from sklearn.model_selection import train_test_split

from sklearn.metrics import accuracy_score


# Load dataset

data = load_iris()

X, y = data.data, data.target


# Split data

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
random_state=42)


# Decision Tree

dt = DecisionTreeClassifier()

dt.fit(X_train, y_train)

dt_pred = dt.predict(X_test)


# Bagging

bag = BaggingClassifier(estimator=DecisionTreeClassifier(), n_estimators=50,
random_state=42)

bag.fit(X_train, y_train)

bag_pred = bag.predict(X_test)


# Accuracy

print("Decision Tree Accuracy:", accuracy_score(y_test, dt_pred))

print("Bagging Accuracy:", accuracy_score(y_test, bag_pred))
```

Sample Output:

Decision Tree Accuracy: 0.93

Bagging Accuracy: 0.97

Question 8: Write a Python program to:

- **Train a Random Forest Classifier**
- **Tune hyperparameters max_depth and n_estimators using GridSearchCV**
- **Print the best parameters and final accuracy**

(Include your Python code and output in the code box below.)

Answer:

```
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import GridSearchCV, train_test_split
from sklearn.datasets import load_breast_cancer

# Load dataset
data = load_breast_cancer()
X, y = data.data, data.target

# Split
X_train, X_test, y_train, y_test = train_test_split(X, y, random_state=42)

# Model
rf = RandomForestClassifier(random_state=42)

# Parameters
param_grid = {
    'n_estimators': [50, 100, 200],
    'max_depth': [None, 5, 10]
}
```

```
# Grid search
grid = GridSearchCV(rf, param_grid, cv=5)
grid.fit(X_train, y_train)

# Results
print("Best Params:", grid.best_params_)
print("Best Accuracy:", grid.best_score_)
```

Sample Output:

Best Params: {'max_depth': 10, 'n_estimators': 100}

Best Accuracy: 0.96

Question 9: Write a Python program to:

- **Train a Bagging Regressor and a Random Forest Regressor on the California Housing dataset**
- **Compare their Mean Squared Errors (MSE) (Include your Python code and output in the code box below.)**

Answer:

```
from sklearn.datasets import fetch_california_housing
from sklearn.ensemble import BaggingRegressor, RandomForestRegressor
from sklearn.tree import DecisionTreeRegressor
from sklearn.model_selection import train_test_split
from sklearn.metrics import mean_squared_error

# Load dataset
data = fetch_california_housing()
X, y = data.data, data.target

# Split
X_train, X_test, y_train, y_test = train_test_split(X, y, random_state=42)
```

```
# Models
bag = BaggingRegressor(estimator=DecisionTreeRegressor(), n_estimators=50)
rf = RandomForestRegressor(n_estimators=50)

# Train
bag.fit(X_train, y_train)
rf.fit(X_train, y_train)

# Predict
bag_pred = bag.predict(X_test)
rf_pred = rf.predict(X_test)

# MSE
print("Bagging MSE:", mean_squared_error(y_test, bag_pred))
print("Random Forest MSE:", mean_squared_error(y_test, rf_pred))
```

Sample Output:

Bagging MSE: 0.25

Random Forest MSE: 0.22

Question 10: You are working as a data scientist at a financial institution to predict loan default. You have access to customer demographic and transaction history data. You decide to use ensemble techniques to increase model performance. Explain your step-by-step approach to:

- **Choose between Bagging or Boosting**
- **Handle overfitting**
- **Select base models**
- **Evaluate performance using cross-validation**
- **Justify how ensemble learning improves decision-making in this real-world context.**

(Include your Python code and output in the code box below.)

Answer:

Step-by-step approach:

1. Choose between Bagging and Boosting

- Use **Bagging** if:
  - Model overfits (high variance)
- Use **Boosting** if:
  - Model underfits (high bias)
- For loan default → Boosting (e.g., Gradient Boosting) is often preferred

---

2. Handle Overfitting

- Use cross-validation
- Limit tree depth
- Regularization (learning rate in boosting)
- Use early stopping

---

3. Select Base Models

- Decision Trees (most common)
- Logistic Regression (baseline)
- Gradient Boosting / XGBoost for better performance

---

4. Evaluate Performance

- Use **k-fold cross-validation**
- Metrics:
  - Accuracy
  - Precision, Recall
  - ROC-AUC (important for imbalance)

---

5. Why Ensemble Helps

- Combines multiple models → reduces risk

- Captures complex patterns in financial data
- More stable predictions
- Better generalization → fewer wrong loan approvals

```
from sklearn.ensemble import GradientBoostingClassifier
from sklearn.model_selection import cross_val_score
from sklearn.datasets import make_classification

# Simulated dataset
X, y = make_classification(n_samples=1000, random_state=42)

# Model
model = GradientBoostingClassifier()

# Cross-validation
scores = cross_val_score(model, X, y, cv=5)

print("Cross-validation scores:", scores)
print("Mean accuracy:", scores.mean())
```

Sample Output:

Cross-validation scores: [0.91 0.92 0.90 0.93 0.91]

Mean accuracy: 0.914